

Autenticação do painel admin

Este documento cobre **só o login administrativo** (`/admin/*`): como ele funciona, o que verifica a identidade do usuário e como as rotas são protegidas. Não cobre nenhum outro sistema do projeto.

Resumo

- Autenticação **por token Bearer** via Laravel Sanctum, no modo token de API — **não** é sessão/cookie.
- Não existe "sessão" no sentido tradicional: cada token é um registro em `personal_access_tokens`, vinculado a um `User`, sem expiração automática configurada (fica válido até ser revogado no logout).
- Só existe um papel: usuário comum vs. `is_admin = true`. Não há níveis intermediários.
- A proteção de verdade é **inteiramente no backend**. Qualquer verificação no frontend é só UX (evitar mostrar a casca do painel pra quem não tem token) — nunca é a barreira de segurança.

Por que Bearer token e não cookie de sessão

`config/cors.php`: `supports_credentials => false` e `allowed_origins => ['*']`. A API aceita requisições de qualquer origem, sem enviar/receber cookies — então autenticação por sessão (que depende de cookie) não é uma opção sem afrouxar o CORS pra todos os outros endpoints públicos. Por isso o login emite um token opaco que o cliente guarda e reenvia manualmente em cada chamada, via header `Authorization`.

Fluxo de login, passo a passo

1. **Frontend** (`src/app/admin/login/page.tsx`) — formulário de e-mail/senha chama `adminLogin(email, password)`.
2. **Backend** — `POST /api/admin/login`, roteado em `routes/api.php` com `throttle:10,1` (10 tentativas por minuto por IP), tratado por `Admin\AuthController::login()`:
 - Busca o `User` pelo e-mail.
 - Valida a senha com `Auth::guard('web')->getProvider()->validateCredentials($user, $data)` — usa o provider de credenciais do guard `web` só para checar o hash da senha; não abre sessão nenhuma nesse guard.
 - Se e-mail/senha não baterem → `ValidationException` (422).
 - Se o usuário existir mas **não** tiver `is_admin = true` → 403, "Este usuário não tem acesso ao painel administrativo." Ou seja: qualquer linha na tabela `users` pode logar, mas só quem tem a flag de admin passa daqui.
 - Se passou nos dois: `$user->createToken('admin-panel')->plainTextToken` — gera um novo Sanctum personal access token e devolve o texto puro **uma única vez** (o Sanctum só guarda o hash no banco).
 - Resposta: `{ token, user: { id, name, email } }`.
3. **Frontend** — guarda o token em `localStorage` (`setAdminToken`, em `src/lib/adminAuth.ts`), sob a chave `votus_admin_token`, e redireciona para `/admin`.

4. **Chamadas seguintes** — `adminService.ts` monta `authHeaders() = { Authorization: "Bearer <token>" }` a partir do token salvo, e injeta esse header em toda chamada a `/api/admin/`.

Como o backend verifica a identidade (a parte que protege de verdade)

Duas camadas de middleware, aplicadas em conjunto em cada rota admin protegida (ver o grupo em `routes/api.php`):

```
Route::middleware(['auth:sanctum', 'admin'])->group(function () {  
    // /admin/logout, /admin/me, /admin/dashboard, /admin/news/*, etc.  
});
```

- **auth:sanctum** (nativo do Sanctum) — lê o header `Authorization: Bearer <token>`, procura o hash correspondente em `personal_access_tokens`, resolve o `User` dono do token e popula `$request->user()`. Sem header, ou com token inválido/revogado → 401. Registrado como guard padrão em `config/auth.php` (`api` → driver `sanctum`).
- **admin** — alias definido em `bootstrap/app.php` para `EnsureIsAdmin`:

```
public function handle(Request $request, Closure $next): Response  
{  
    if (! $request->user()?->is_admin) {  
        return response()->json(['message' => 'Acesso restrito a administradores.'], 403);  
    }  
    return $next($request);  
}
```

Roda **depois** de `auth:sanctum` (que já resolveu `$request->user()`), e simplesmente checa a coluna `is_admin` do usuário autenticado. Sem ela → 403.

Ou seja: para acessar qualquer rota dentro desse grupo, a requisição precisa (1) trazer um token Sanctum válido e (2) o dono do token precisar ter `is_admin = true` no banco. As duas condições são checadas a cada requisição, não só no login — não existe nada como um "cargo" embutido no token; o token só identifica o usuário, e o `is_admin` é sempre lido do banco em tempo real.

Detalhe de configuração relevante em `bootstrap/app.php`:

```
// API pura, sem tela de login web: sem isso, o middleware "auth:sanctum"  
// tenta redirecionar pra uma rota "login" inexistente sempre que o  
// cliente não manda um Accept: application/json explícito, e isso vira  
// 500 em vez de 401.  
$middleware->redirectGuestsTo(fn () => null);
```

Sem essa linha, uma chamada sem token que não declarasse `Accept: application/json` quebrava com 500 em vez de simplesmente devolver 401.

Logout

`POST /api/admin/logout` (mesmo grupo `auth:sanctum` + `admin`) chama `$request->user()->currentAccessToken()->delete()` — apaga só o token usado naquela requisição do banco. Ele para de funcionar imediatamente para qualquer chamada futura. O frontend, em paralelo, limpa o `localStorage` (`clearAdminToken()`).

Não há expiração por tempo: um token emitido continua válido indefinidamente até alguém deslogar (ou até ele ser apagado manualmente do banco). Não existe endpoint de "revogar todos os tokens" nem refresh token.

`/admin/me`

`GET /api/admin/me` (mesmo grupo) devolve `{ id, name, email }` do usuário por trás do token atual. Não é usado para autenticar nada — é usado pelo frontend só para confirmar que um token salvo ainda é válido (ver abaixo).

O que o frontend faz (e por que isso NÃO é a proteção real)

`src/app/admin/layout.tsx` envolve todas as rotas `/admin/*` (exceto `/admin/login`) com uma verificação client-side:

1. Sem token no `localStorage` → redireciona para `/admin/login`.
2. Com token → chama `getAdminMe()`; se der erro (token inválido/expirado/revogado), redireciona para `/admin/login`; se der certo, libera a tela.

Isso existe só para não desenhar a casca do painel (menus, formulários) para quem não tem sessão — é uma conveniência de UX. **A barreira de segurança de verdade é o backend rejeitar a chamada** (401/403) em cada rota `/api/admin/*`, via `auth:sanctum` + `admin`. Um atacante que ignore o frontend inteiramente e chame a API direto esbarra exatamente nas mesmas duas checagens.

Para quem for proteger novas rotas

Para que uma nova rota do backend exija login de admin, basta colocá-la dentro do grupo já existente:

```
Route::middleware(['auth:sanctum', 'admin'])->group(function () {
    Route::get('/admin/minha-rota-nova', [MeuController::class, 'metodo']);
});
```

Não é necessário (nem existe) nenhum outro mecanismo de proteção de rota no projeto — nem policies, nem roles adicionais, nem gates. É só essa combinação de dois middlewares.